

2110531 - Data Science and Data Engineering Tools

NoSQL

Asst. Prof. Natawut Nupairoj, Ph.D.

Department of Computer Engineering, Chulalongkorn University

natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Traditional Database

- Based on ”relational model”
  - Data is split and stored into tables
  - Tables can be processed together using set-like operations
  - Data model is usually normalized to remove duplication
- Very suitable for OLTP or transaction systems
  - Provide lots of complicated SQL operations
  - Lots of inserts and updates

Advertisement


Advertisement



Add Photo







Prep Time:	Total Time:	Servings:
20 mins	20 mins	6

Jump to Nutrition Facts

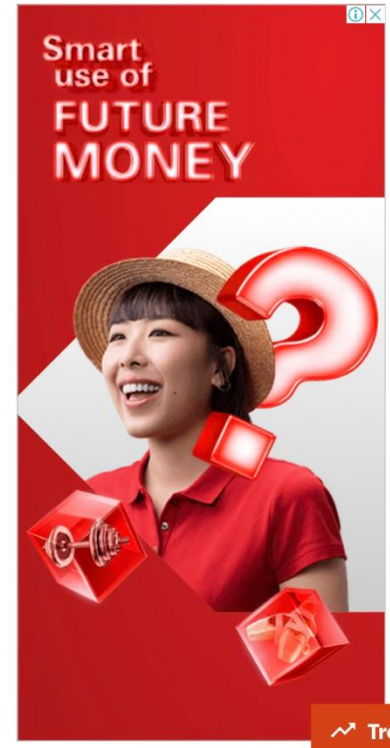
Ingredients

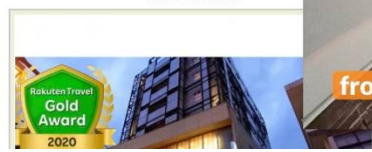
- 3 cloves garlic, peeled
- 3 Thai green chiles
- 6 green beans, cut into 1-inch pieces
- 1 large unripe papaya, peeled and cut into

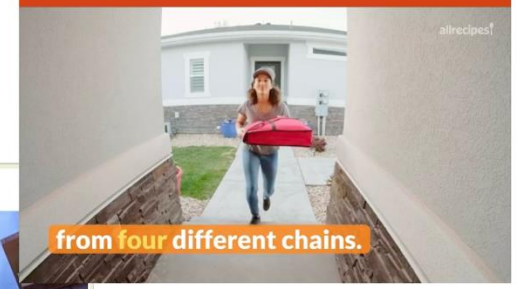
Local Offers

00000 Change

Oops! We cannot find any ingredients on sale near you. Do we have the correct zip code?

Advertisement


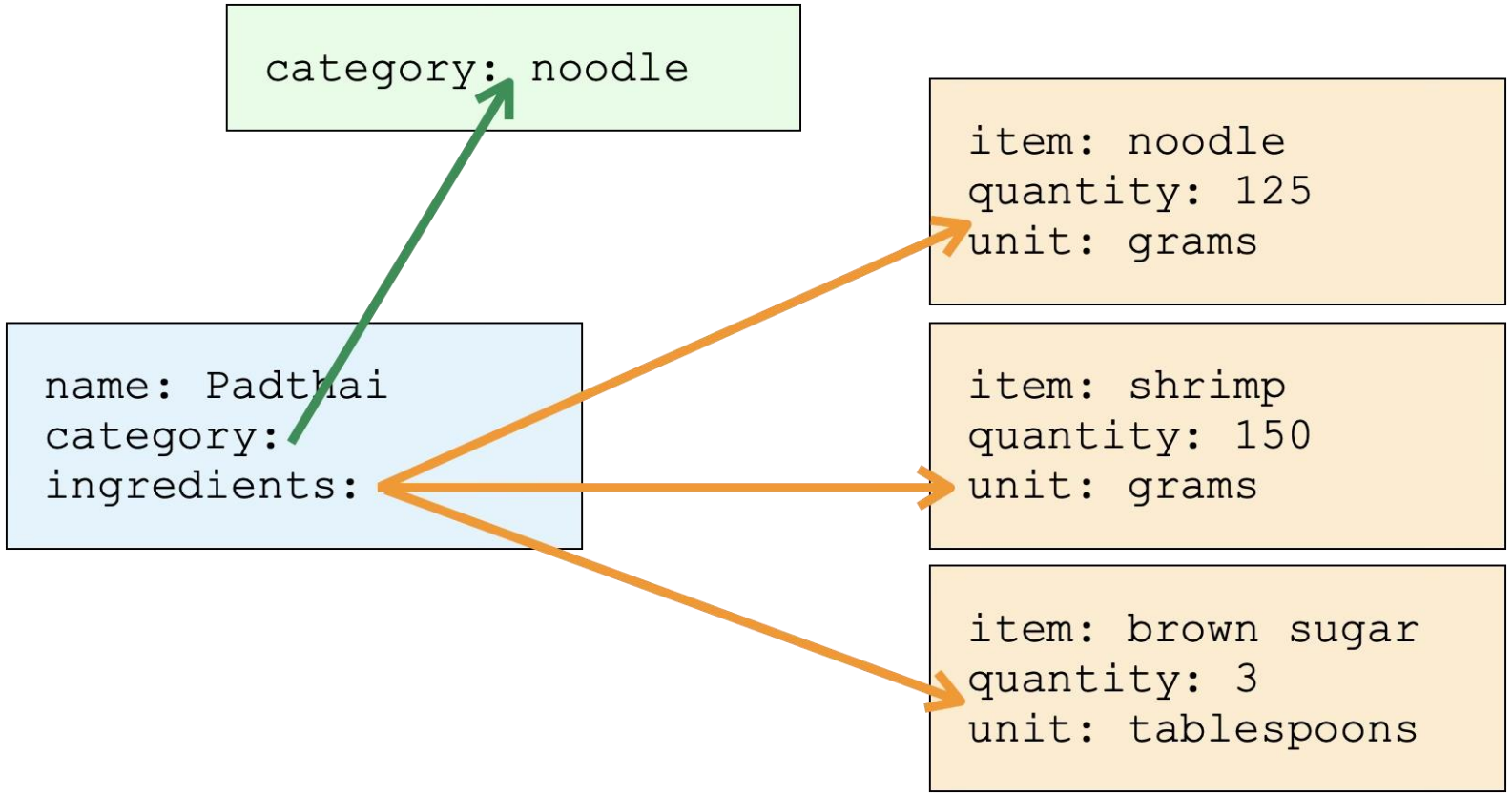
Advertisement
Advertisement


Trending Videos


CHULA ENGINEERING
Foundation toward Innovation

Recipe Data Example

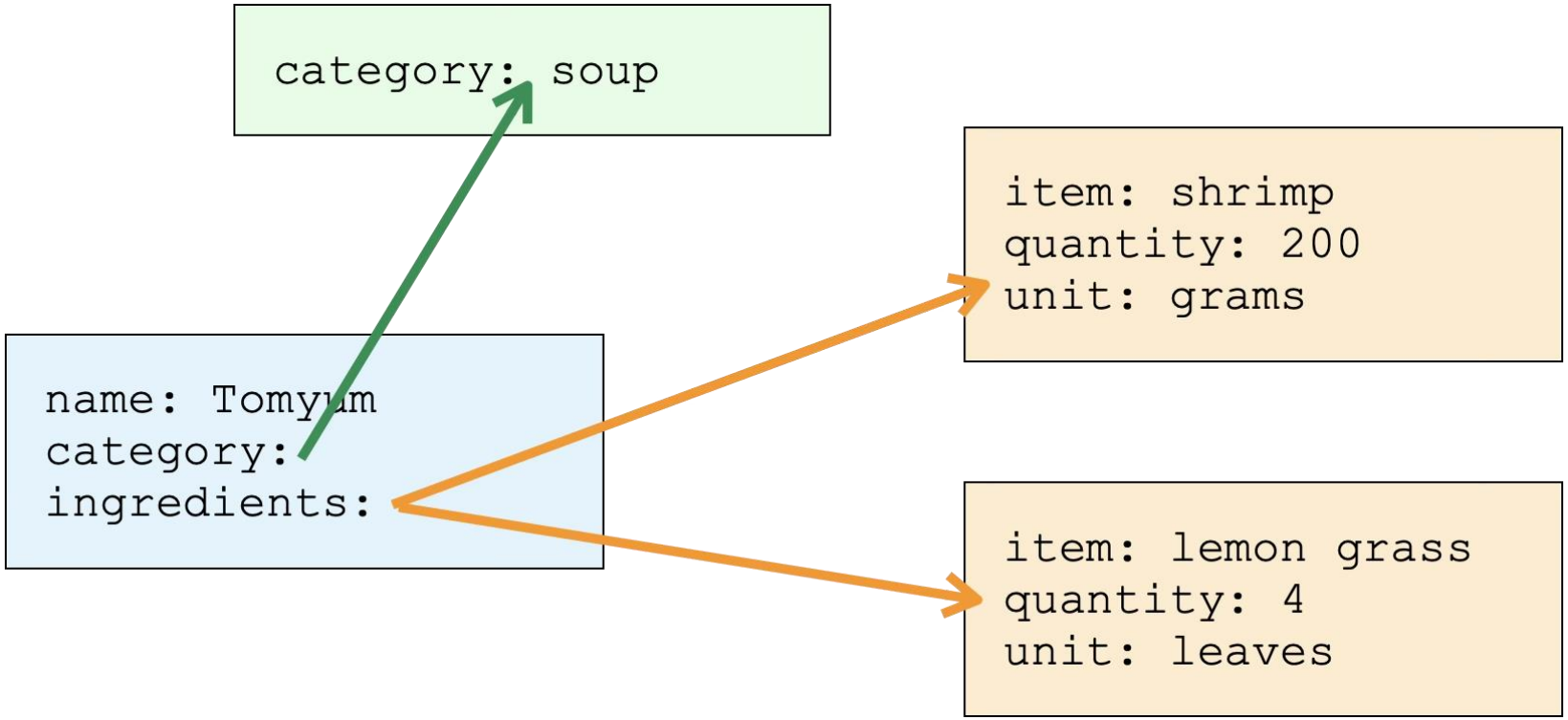
- Padthai belongs to a noodle category with 3 main ingredients noodle, shrimp, brown sugar



CHULA ENGINEERING
Foundation toward Innovation

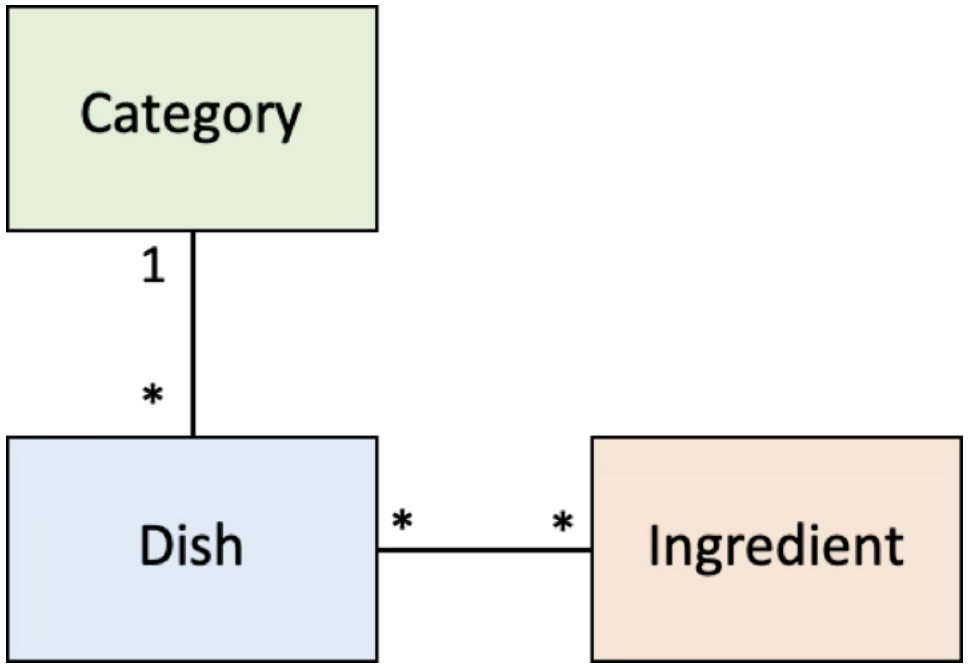
Recipe Data Example

- Tomyum belongs to a soup category; shrimp and lemon grass are main ingredients



Recipe Data is Hierarchical

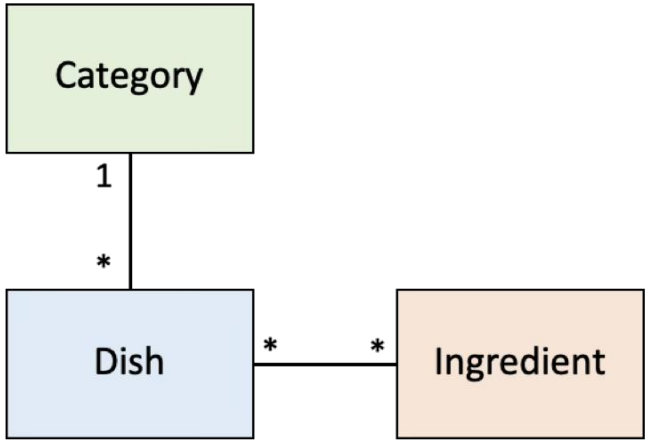
- A dish has the following data:
  - belong to a category
  - consists of one or more main ingredients
  - Ingredients are different in portions and units



Example RDBMS: Recipe

Category

id	category
1	Soup
2	Noodle
3	Rice



Dish

id	name	category	…
1203	Pad Thai	2	
1288	Chicken Rice	3	

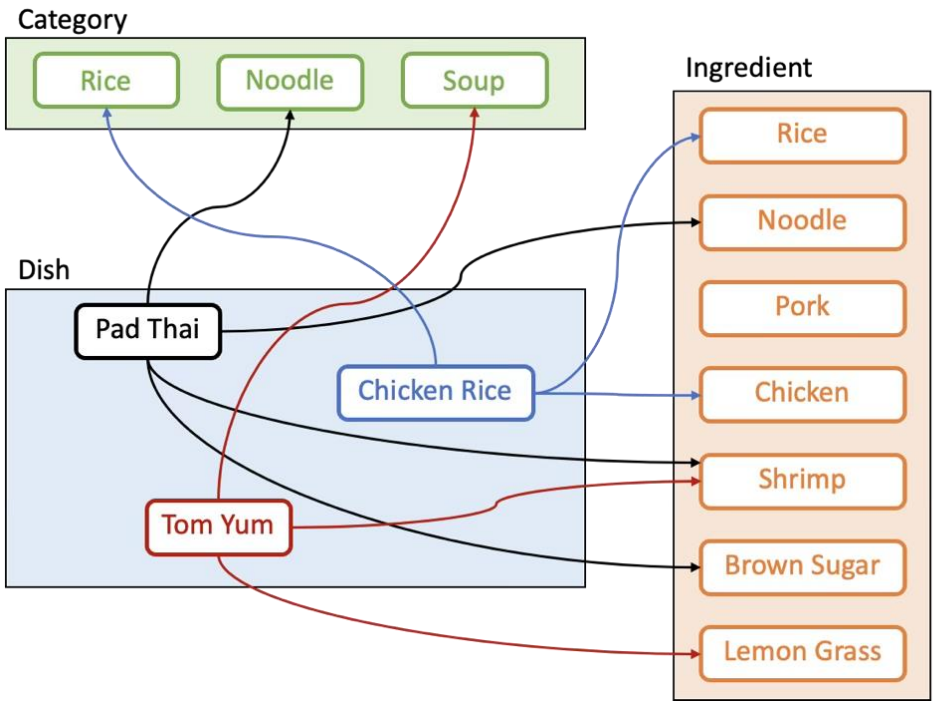
DI_link

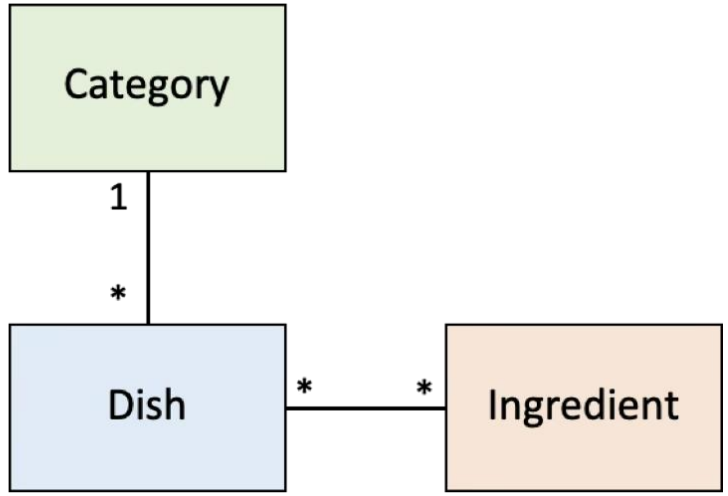
d_id	i_id	quantity	unit
1203	45	125	g
1203	48	150	g
1203	52	3	tbsp

Ingredient

id	item
45	noodle
46	rice
48	shrimp
52	brown sugar

Relational Database: Normalized Data Model





- How can we split these tables to lots of machines?
  - Dishes of the same category in the same machine
  - How about dishes and ingredients?
- Or we can replicate data — lead to data consistency problems

Problems of Data Science Storage

- There are several needs for data analytics purposes e.g. traditional data store, caching, feature store
- Data is historical data and its volume can be huge
- Scalability is extremely important and “Relational + Consistency” can limit scalability
- SQL command can be very complex and time-consuming
  - It requires the synchronization of data accessing between multiple tables
  - It will be poor when using on more than a few servers in the same cluster

CAP Theorem (Brewer's Theorem)

- By Eric Brewer (University of California, Berkeley)
- It is impossible for a distributed computer system to simultaneously provide all three of the following guarantees: Consistency, Availability, Partition tolerance
- When the communication failure occurs, the system has to choose either consistency or availability

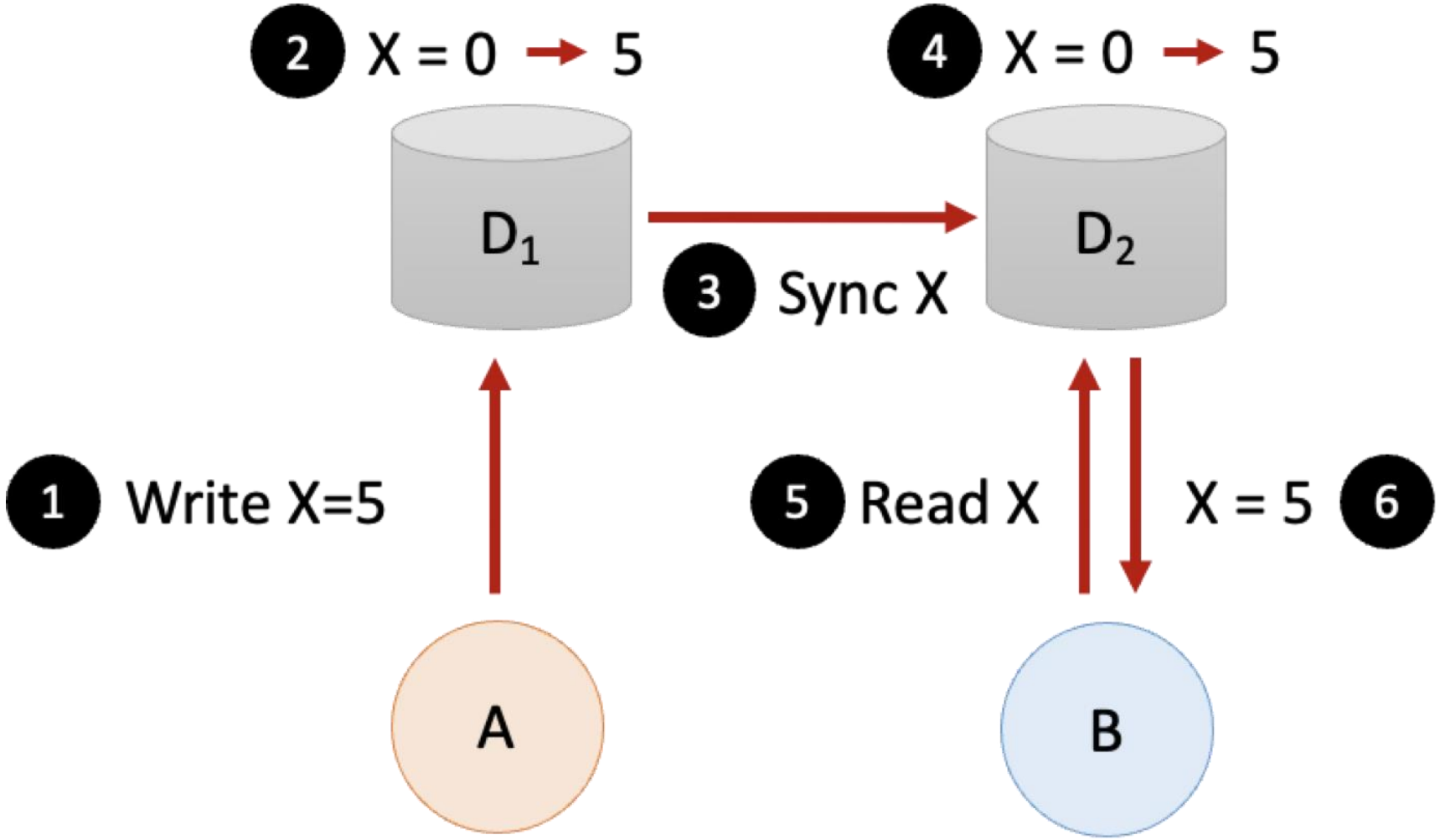
CAP Theorem Scenario

- Distributed system (clients and servers)
- Multiple servers working together
- Multiple clients may read or write on the same data at the same time

CAP Guarantees only 2 out of 3

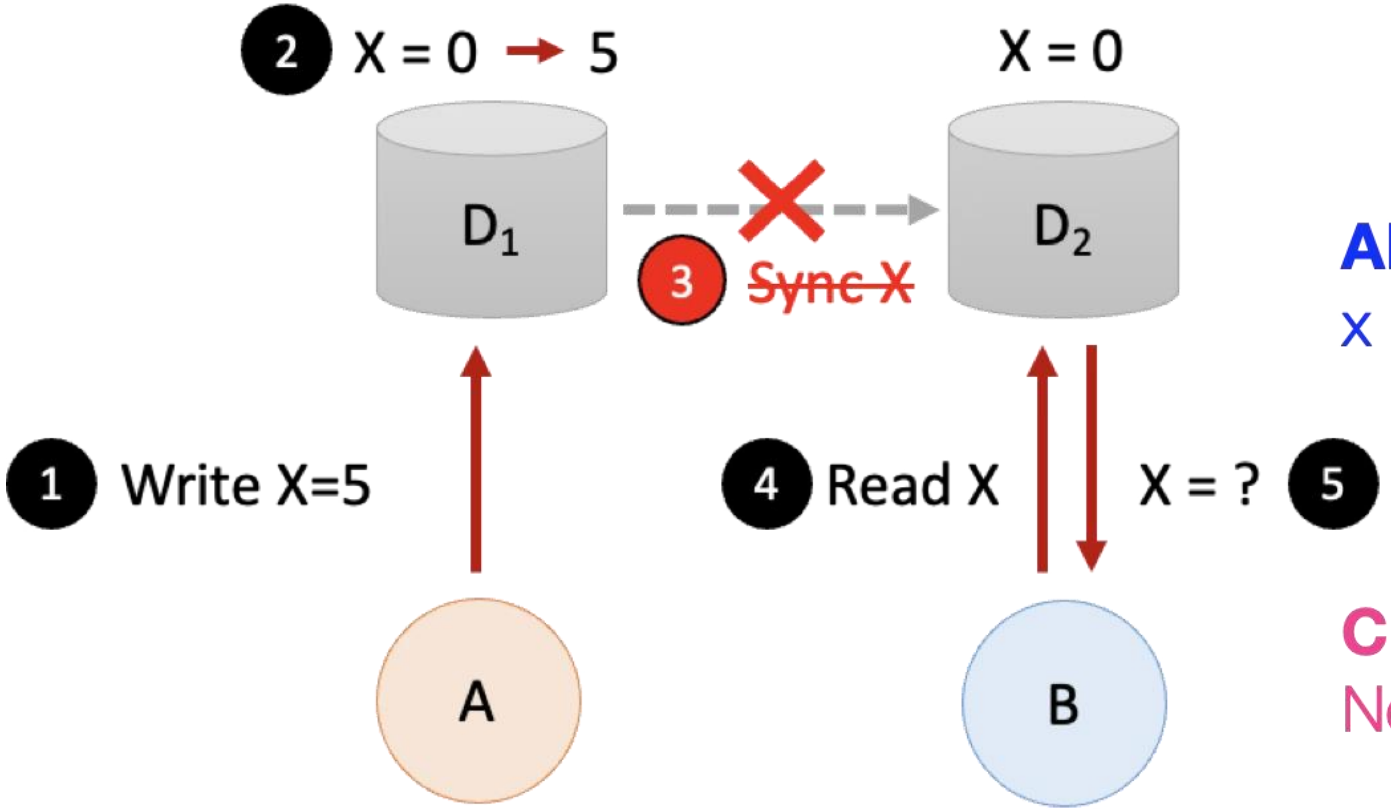
- Consistency
  - Every read receives the most recent write or an error
- Availability
  - Every request receives a (non-error) response – without guarantee that it contains the most recent write
  - Response from any server is good
- Partition tolerance
  - The system continues to operate despite arbitrary message loss or failure of part of the system
  - Lots of servers require long synchronization time causing servers to not be able to communicate among one another within reasonable time

CA Data Store



CHULA ENGINEERING
Foundation toward Innovation

Given P, Choosing between C and A (at step 5)



AP Data Store

x = 0 (not consistent, but available)

CP Data Store

No response

More Scenario - when partition occurs

- CA - cannot operate
- CP
  - allow read requests and reject all write requests
  - allow read and write at a group that still connects to a master node

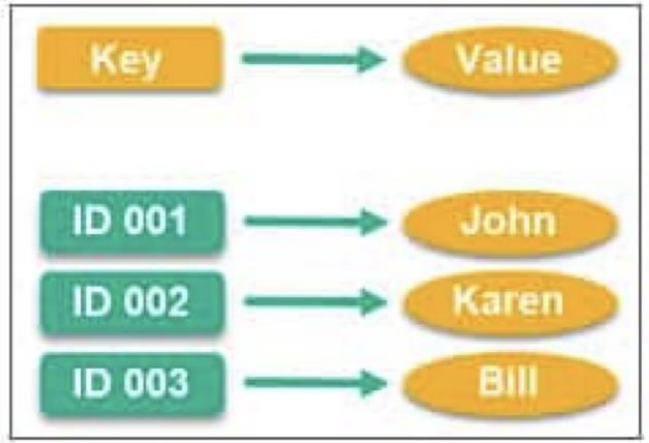
The Landscape of NoSQL

- Alternatives to SQL database
  - non-relational
  - distributed
  - horizontally scalable
- Data is shared and distributed across multiple servers
- Typically use weak consistency model (but not always)

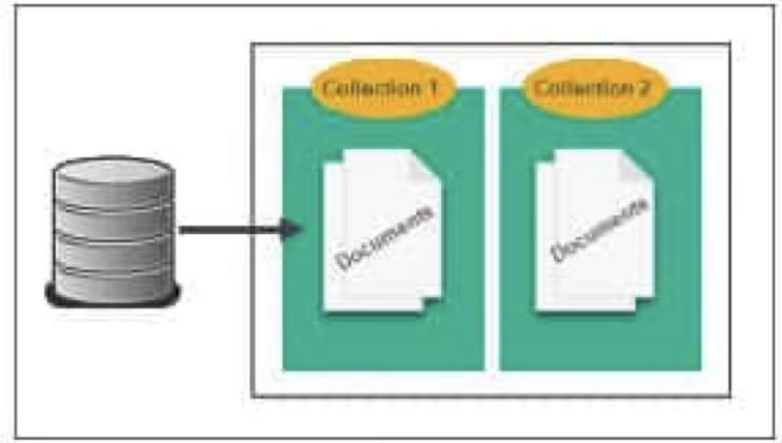
Types of NoSQL

- Document: MongoDB, DynamoDB, CosmosDB, Couchbase, Firebase
- Column: Cassandra, HBase, CosmosDB, Accumulo
- Key-value: Redis, DynamoDB, CosmosDB, MemcacheDB
- Graph: Neo4J, TigerGraph, ArrangoDB, OrientDB
- Search Engine: Elasticsearch, Splunk, Solr
- Timeseries: InfluxDB, TimescaleDB, Prometheus, Graphite
- Vector: Chroma, Pinecone

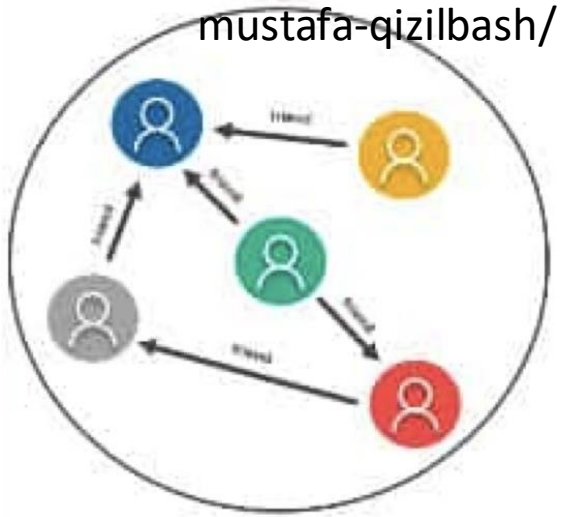
Key-value



Document



Graph



Wide-column

Source: https://www.linkedin.com/pulse/sql-nosql-databases-mustafa-qizilbash/

Row-oriented

ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented

Name	ID
John	001
Karen	002
Bill	003

Grade	ID
Senior	001
Freshman	002
Junior	003

GPA	ID
4.00	001
3.67	002
3.33	003

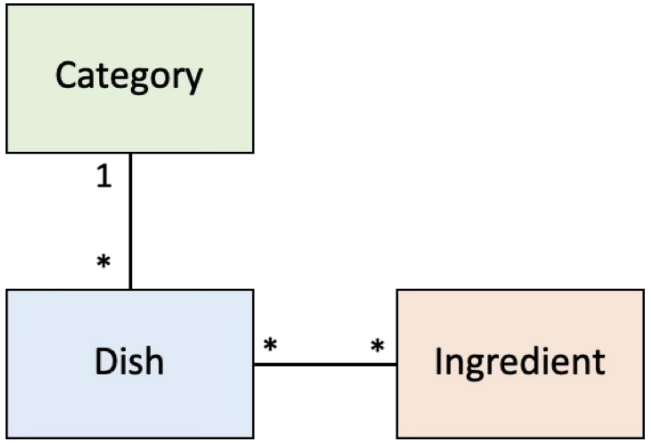
How NoSQL can “Scale”

- Principle ideas
  - Split data into chunks or shards
  - Distribute data across multiple servers
  - Must require minimum synchronization
- Have to give up some traditional features
  - No complex relational model
  - Relax consistency
  - Duplicated information (not space optimized)
  - Fast to insert new record, but not so fast to update the existing one

Example RDBMS: Recipe

Category

id	category
1	Soup
2	Noodle
3	Rice



Dish

id	name	category	…
1203	Pad Thai	2	
1288	Chicken Rice	3	

DI_link

d_id	i_id	quantity	unit
1203	45	125	g
1203	48	150	g
1203	52	3	tbsp

Ingredient

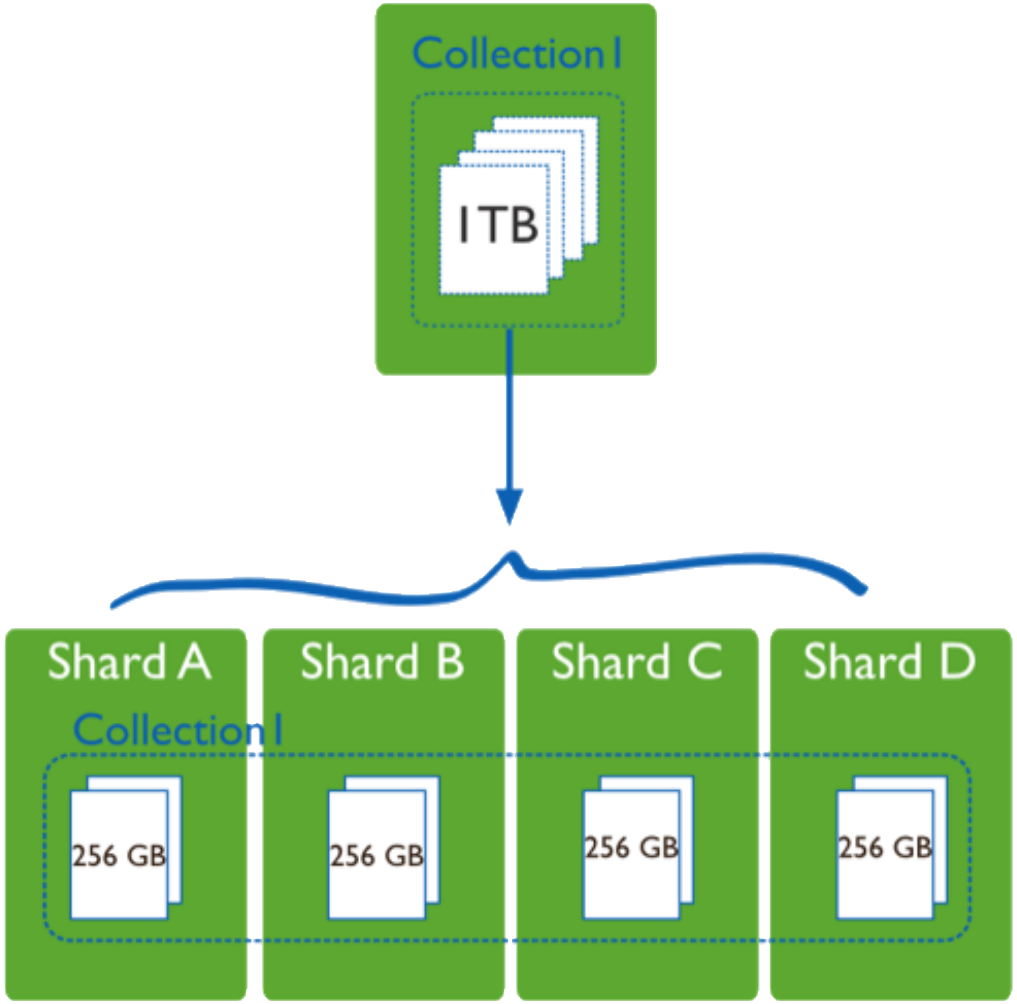
id	item
45	noodle
46	rice
48	shrimp
52	brown sugar

Example MongoDB: Recipe

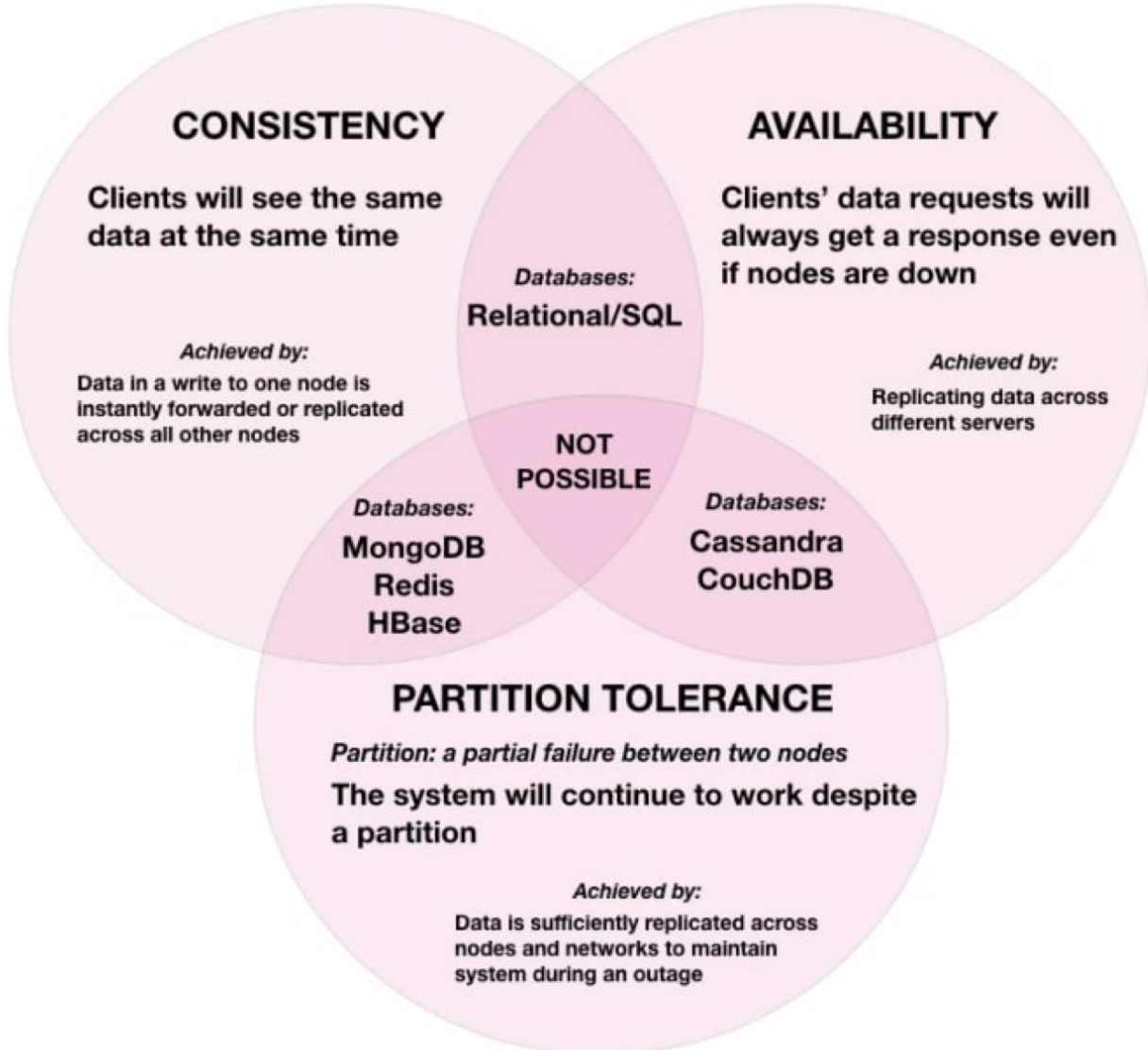
```
{
  _id: ObjectId ("507f191e810c19729de860ea""),
  name: "Pad Thai",
  cuisine: "Thai",
  category: "Noodle",
  rating: 4.7,
  votes: 9301,
  ingredients: [
     { item: "noodle", quantity: "125", unit: "g" },
     { item: "shrimp", quantity: "150", unit: "g" },
     { item: "brown sugar", quantity: "3", unit: “tbsp" }
  ]
}
```

CHULA ΣNGINEERING
Foundation toward Innovation

How MongoDB Scales - Sharding



- Large-Scale MongoDB bases on Horizontal Scaling mechanism
- Storing shards (blocks of data) across multiple machines
- Data partitioned into shards with shard keys
- Each shard handles only operations related to its block



Source: https://dev.to/katkelly/cap-theorem-why-you-can-t-have-it-all-ga1